

Game Proposal: Bark Brawlers

CPSC 427 – Video Game Programming

Team: 4

Ahren Spadinger-Fengler (74156449)

Ricky Wong (98680127)

Kai Chen (65392664)

Ryan Tan (66458373)

Story:

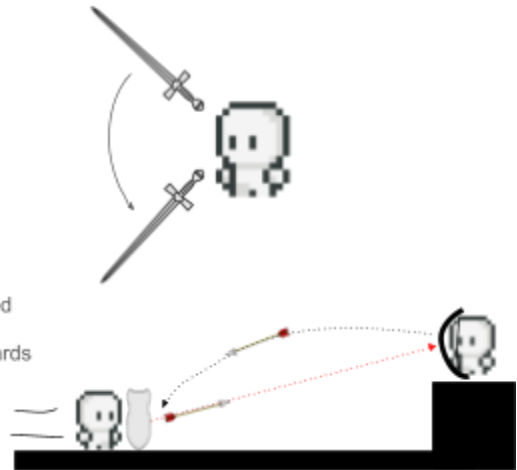
Briefly describe the overall game structure with a possible background story or motivation. Focus on the gameplay elements of the game over the background story.

The 4 mighty (anthropomorphic) dogs of the realm all want the title of TOP DOG. They have decided to settle this through a series of short battles. Bark Brawlers is a 2d Fantasy platform fighter in a pixel art style. It is focused on 1v1 gameplay (local multiplayer) where players can control their characters with the keyboard (or possibly the gamepad, see advanced technical features section). Players fight in an enclosed arena, and combat is health based; first to drop to 0 health loses. There are 4 classes: the Knight, Mage, Archer, and Thief. Each character and move/aim (8 directions), jump (with variable jump height), attack (with the option to hold the attack button to do a powerful charged attack), and use a special ability (only after successfully landing a previous attack; a hit must be landed before each time the special ability is used). Matches consist of 1-2 rounds, with the first to 5 victories winning. Each round players are put into a random map from a small pool of arenas.

Classes:

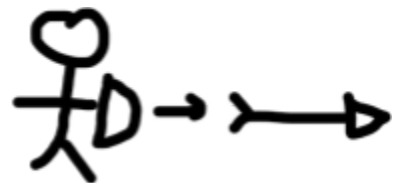
KNIGHT

- Attack: Sword
 - Large sweep motion
 - Charging up = bigger sweep and more damage.
- Special: Shield bash
 - Raises a shield and dashes left or right (no up/down)
 - Any opponents in the way get hit and knocked back are stunned for half a second
 - Any incoming projectile attacks are reflected back straight towards the attacker, with no gravity applied
- Passive:
 - More hp



ARCHER

- Attack: Arrow
 - Arrow, has zero gravity for a set distance before rapidly falling
 - Charging up = Faster arrow speed, further travel, and more damage.
- Special:
 - Quick jump back, and fire an arrow toward the original location
 - Attack coming from direction facing are absorbed
- Passive:
 - Higher jump

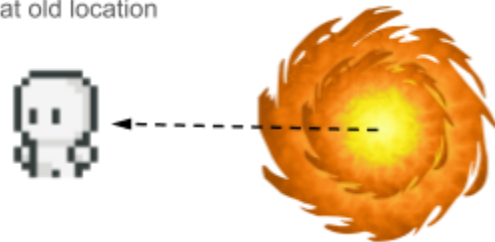


MAGE

- Attack: Fireball
 - Launch a fireball that ricochets off walls
 - They decay over time (mid range projectile)
 - Charging up = Larger projectile, longer range, and more damage.



- Special: Explosive Teleport
 - Can teleport a short distance in 4 directions (up, left, down, right)
 - Leaves behind an explosion that deals damage at old location
 - The explosion destroys projectiles



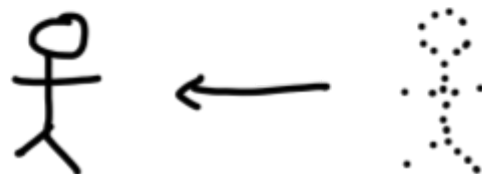
- Passive:
 - Faster walking speed

THIEF

- Attack: Dagger slash/throw
 - Slash with dagger (short range)
 - Charging up = Throw a dagger (short range)
 - Only works if attack is fully charged



- Special:
 - Blink forward a set distance (no hurtbox)
 - Can blink in any 8 directions

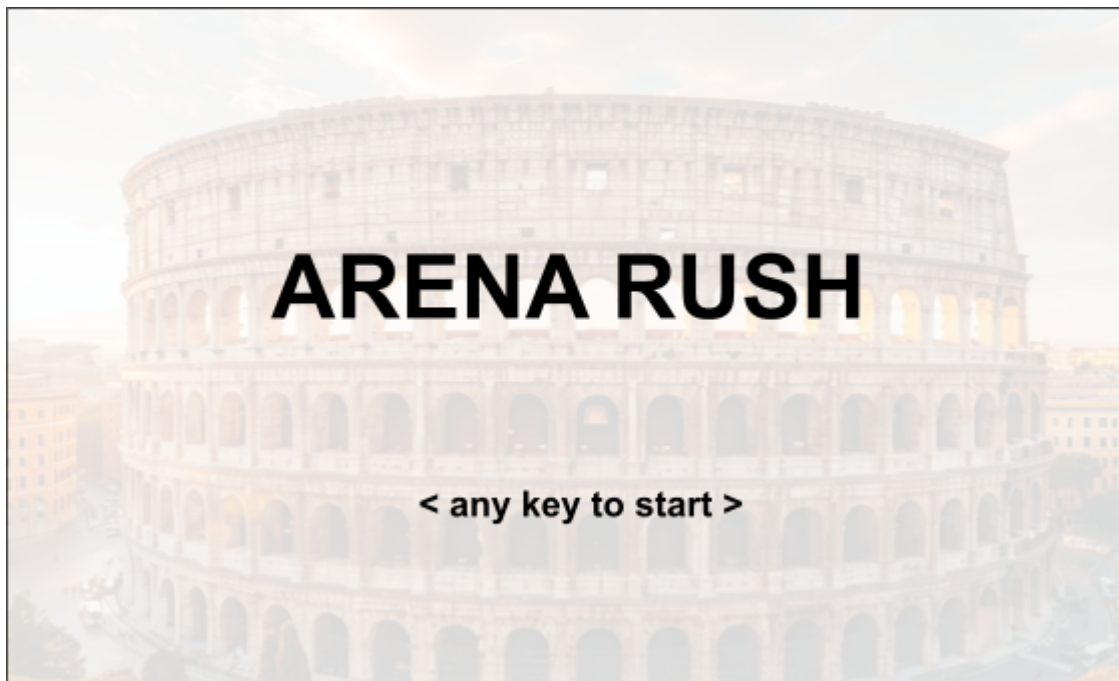


- Passive:
 - Gets an extra jump in the air

Scenes:

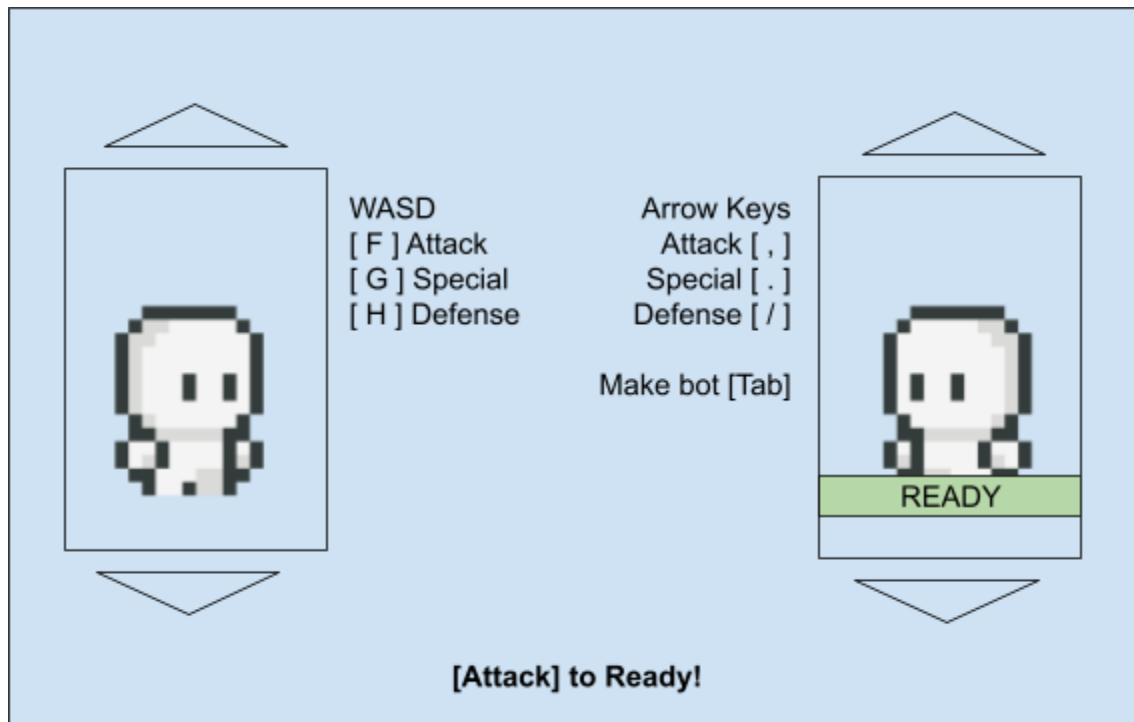
Produce basic, yet descriptive, sketches of the major game states (screens or scenes). These should be consistent with the game design elements, and help you assess the amount of work to be done. These should clearly show how players will interact with the game and what the outcomes of their interactions will be. For example, jumping onto platforms, shooting projectiles, enemy pathfinding or 'seeing' the player. This section is meant to demonstrate how the game will play and feeds into the technical and advanced technical element sections below. If taking inspiration from other games, you can include annotated screenshots that capture the game play elements you are planning to copy.

Title:



Character Select:

Players can choose their characters using the Up/Down keys, and ready for battle using the [Attack] key. Once both players are ready, the first round of combat begins.



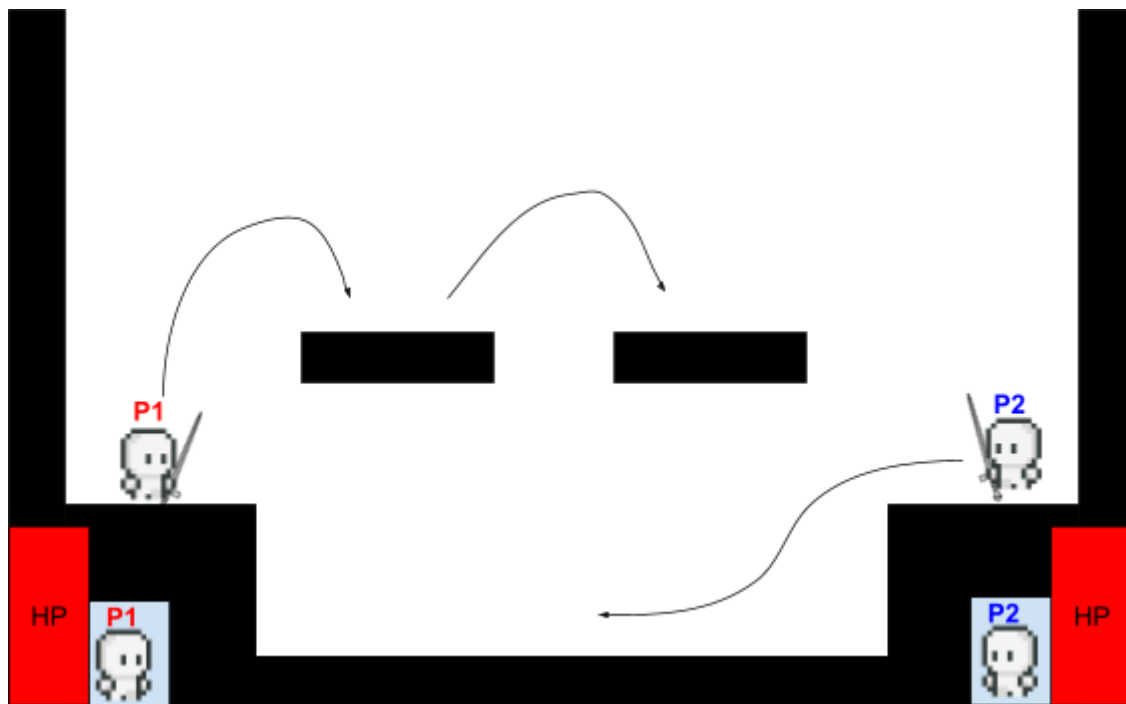
Arena:

- Art style:

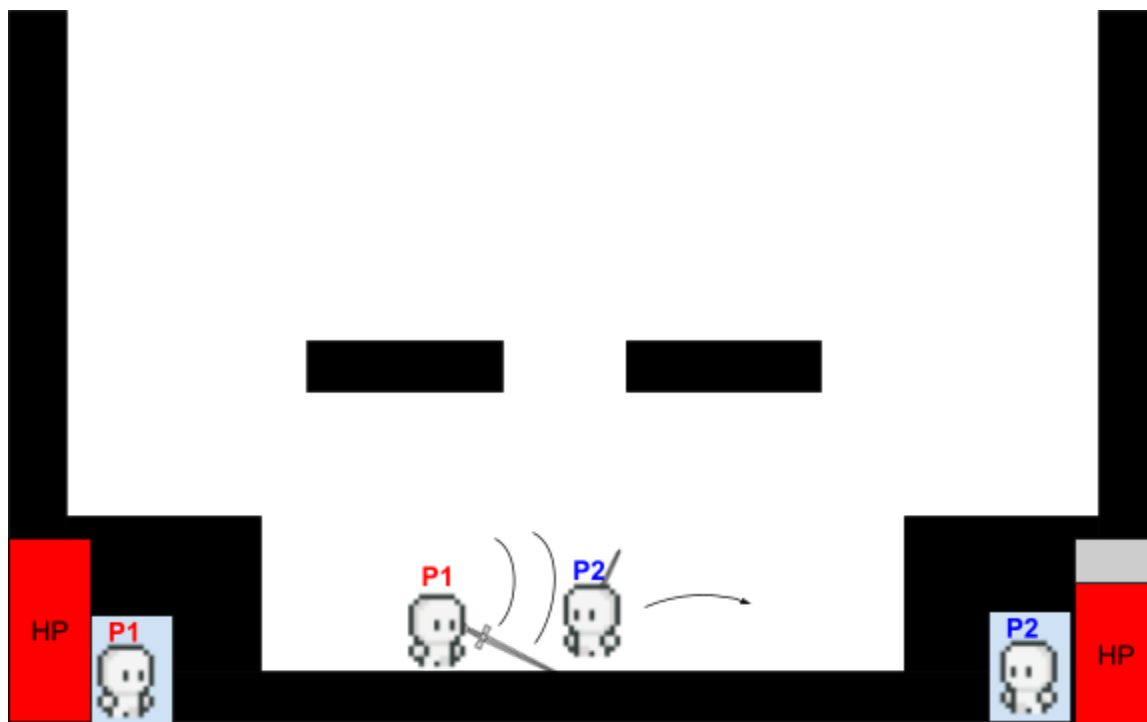


- Layout:

Player movement

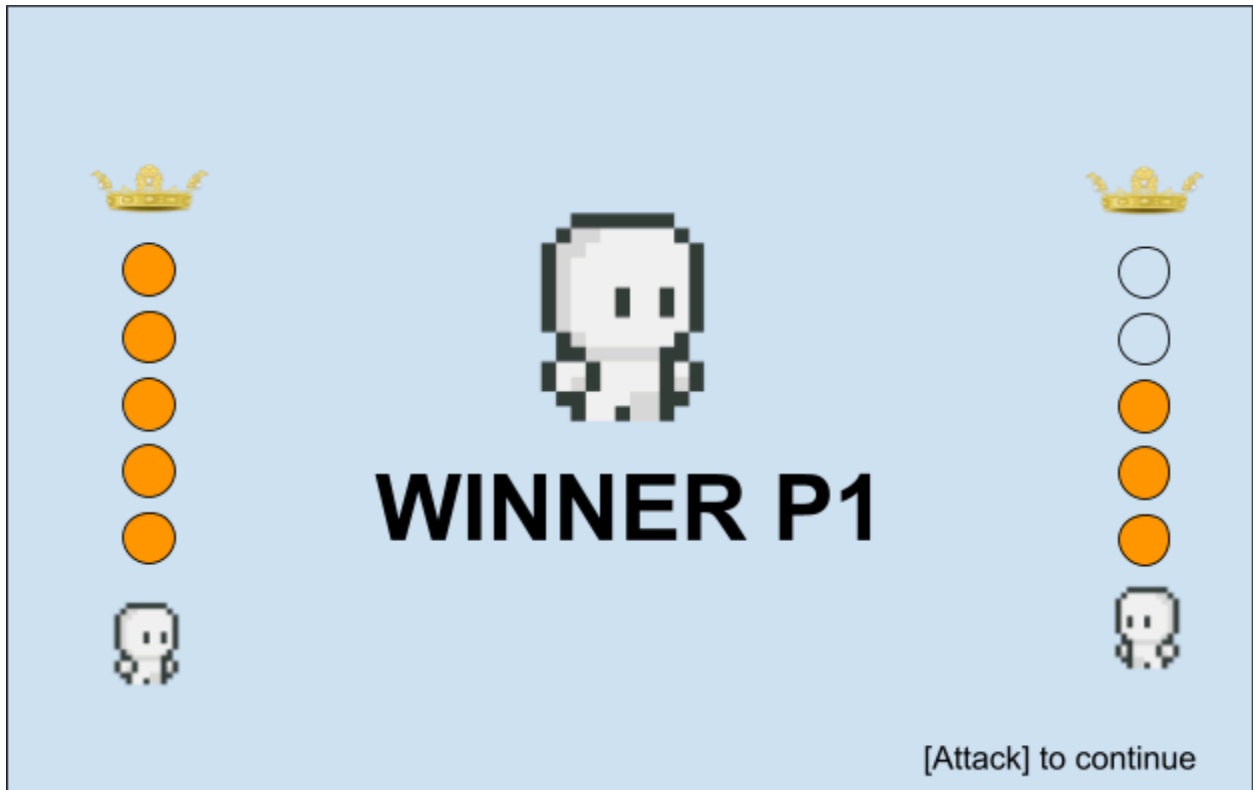


P1 swings their sword to the right and hits P2



End Game Screen:

Player 1 has won 5 rounds, Player 2 has won 3 rounds.



Technical Elements:

Identify how the game satisfies the core technical requirements: rendering; geometric/sprite/other assets; 2D geometry manipulation (transformation, collisions, etc.); gameplay logic/AI, physics.

Rendering:

- 2D camera view (simple rendering effects)
- Particle system for attacks, visual effects when getting hit

Geometric / Sprite / Other Assets:

- Sprites for character animations (walk/run/attack), arena backgrounds
- Background music, sounds for attacking/specials/getting hit

2D Geometry Manipulation:

- Player movement, hitbox collision system

Gameplay Logic

- Health-based combat system

AI

- Option to set P2 to a CPU Knight.
- Simple path finding

Physics:

- Platformer physics (basic 2D physics)

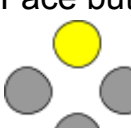
Advanced Technical Elements:

List the more advanced and additional technical elements you intend to include in the game prioritized on likelihood of inclusion. Describe the impact on the gameplay in the event of skipping each of the features and propose an alternative.

1. Gamepad Support, alternative is existing keyboard controls
2. 2D dynamic shadows, alternative is static shadows or no shadows
3. Numerous sophisticated integrated assets: music that changes as health gets lower; visually coherent sprites for the playable characters and arena, alternative is basic assets

Devices:

Explain which input devices you plan on supporting and how they map to in-game controls.

	Keyboard (P1)	Keyboard (P2)	Gamepad (P1 or P2)
Move/Aim	Arrow Keys	WASD	L Stick or D Pad
Jump	,	F	Face button: 
Attack	.	G	Face button: 
Special	/	H	Face button: 

Tools:

Specify and motivate the libraries and tools that you plan on using except for C/C++ and OpenGL.

Tools:

- Piskel for sprite and map design
- Audacity for sound editing
- Desmos for math and creating animation curves.

Libraries:

- SDL: audio, input (e.g. gamepad)
- GLFW: windowing, input handling

Team management:

Identify how you will assign and track tasks and describe the internal deadlines and policies you will use to meet the goals of each milestone.

Tasks will be assigned so the following people are working in the following areas.

1. Engine: Ahren, Ryan, Ricky, Kai
2. Gameplay: Ahren, Ryan
3. UI: Kai, Ricky
4. Assets: Ahren, Ricky, Ryan
5. Level Design: Ahren, Ryan
6. Gameplay Design: Ahren, Ryan
7. QA: Kai, Ricky

Each week when we meet we will sort tasks by importance and time-estimate. We will assign the most important and quickest tasks first, before moving onto the other tasks. In order to avoid missed deadlines, we will go over everyone's availability for the week and assign less work to those who are busy. If there is a pattern of certain people constantly being busy, we will discuss it in meetings, before possibly taking further action. If deadlines are still expected to be missed teammates should notify the team at least 3 days ahead, and 5 days ahead for longer features.

Most tasks will be due at the end of the week, except on Milestone weeks in which tasks will be due by **Sunday 8am** for merging. Some exceptions may be made for certain tasks for which other tasks depend on: this will cause deadlines to be earlier (this will mainly be an issue near the beginning of development, while the engine is still being fleshed out). In this case deadlines and task assignments may rely more on teammate availability rather than specified roles or preferred tasks.

We will be using separate feature branches and block committing directly to the main branch. Merging code will not usually require a review, but in the case of members collaborating on a feature, or a large feature that may cause conflicts needs to be merged, we will ask for a review from another member as needed.

Development Plan:

Provide a list of tasks that your team will work on for each of the weekly deadlines. Account for some testing time and potential delays, as well as describing alternative options (plan B). Include all the major features you plan on implementing (no code).

In general, implementation priority is from top to bottom for each category, while following the weekly plans for each milestone. In the event in which a task is not completed for that week, we will shift it for the following week. Items listed with an exclamation point (!) is deemed as a core

feature and will be given priority each week. Our plan B is that non (!) may be dropped if there is not enough time.

Overview of features (see next section for timeline):

Asset Management:

- (!) Sprite loading / drawing
- (!) Sound loading / playing

Input handling:

- (!) Keyboard
- Gamepad

ECS:

- (!) Create / remove entities
- (!) Add / remove components
- (!) Be able to iterate over all entities with a certain component

Platformer Player Controller:

- (!) Collision with walls
- (!) Gravity
- (!) Walk acceleration / stopping
- Coyote Time
- Variable jump height
- Edge clipping

Hitbox/Hurtbox System

- (!) Hitbox / hurtbox collisions
- (!) Collision layers / masks

Level / Arena Rendering System:

- (!) Loading from PNG file (1 pixel = 1 tile; colors map to different kinds of blocks)
- 47 Auto tiling

Combat System:

- (!) Apply impulses after hit
- (!) Hit stun
- (!) Hit freeze
- (!) Cooldown

RoundManager:

- (!) Score tracking

Characters Abilities:

- (!) Knight Character:
 - Sword swing (8 directions)
 - Variable sword swing (bigger swing for more charge)
 - Shield Dash
 - Shield Dash Reflected Projectile
 - +HP passive

- (!) Archer Character:
 - 8-direction Arrow projectile (with gravity)
 - Variable arrow projectile (charge for straighter trajectory)
 - Quick jump back
 - (fire arrow towards original location)
 - Absorb/remove enemy projectile from original jump back location
 - +jump height passive

- (!) Mage Character:
 - Fireball projectile (no gravity)
 - Linear decay in damage & size
 - Ricochet off walls
 - Variable Fireball Projectile (larger projectile)
 - 4-direction blink + combustion projectile (knockback entities)
 - +speed passive

- (!) Rogue Character:
 - Dagger slash
 - Short range slash with dagger
 - Dagger throw
 - Charge up and throw a dagger (short range) / critically
 - Only works if fully charged
 - Blink forward a set distance with no hurtbox
 - +extra jump passive

Sound System

- BGM
- sound effects on hit
- sound effect on jump

Menu System

- (!) Splash / Start screen
- (!) Character Select + Control information
 - Description for each character

- Move set description
- Fighter compendium
- End screen

GUI

- (!) Health / Hearts
- (!) Skill / key cooldown
- (!) Player/Character identification (player 1, player 2)
 - Label above character
 - (icon / splash art? on bottom left/right)
- (!) Round timer
- (!) Scoreboard
- Screenshake

UI/UX | Gameplay | Engine | Assets

Milestone 1: Skeletal Game

Goal: Have a character that can move around, load placeholder sprites, load level (platform blocks), load arena art (tileset)

Goals: ECS, Sprite Drawing, Input Handling, Hitbox / Hurtbox, Platformer Controller, Level Loading, Tileset

Week 1: ECS, Sprite Drawing, Input Handling,

Week 2: Hitbox / Hurtbox, Platformer Controller Level Loading, Tileset

Milestone 2: Minimal Playability

Goal: In game UI, Death / Multiple Arenas, Knight Character, Hit Stun, Knockback, Health, Knight Character

Week 1: Knight Character, In game UI, Health, Death

Week 2: Multiple Arenas, Hit Stun, Hit Freeze, Knockback

Milestone 3: Playability

Goal: Archer Character, Mage Character, Rogue Character, Character Select, End Game Screen, Arena Backgrounds

Week 1: Archer Character, Mage Character, Rogue Character, End Game Screen

Week 2: Character Select, Arena Backgrounds

Milestone 4: Final Game

Goal: Balance, Particle Effects, Sound Effects, Hit Flash Screenshake

Week 1 Particle Effects, Sound Effects, Hit Flash

Week 2 Balance, Screenshake